Arweave for the BANKON stack — a senior architect's deep-dive

Arweave is **not** a chain you deploy Solidity to; it is a permanent, content-addressed storage substrate whose native programmability now lives in **AO**, an actor-model hyper-parallel computer that runs Lua processes and persists every message to Arweave. As of May 2026, AR trades around **$1.64–$2.40** (~$2 central), the protocol is on **2.9.x with `replica.2.9` packing**, **AO mainnet has been live since Feb 8, 2025**, **HyperBEAM** is rolling out as the production AO-Core implementation, **Bundlr/Irys has pivoted off Arweave to its own EVM datachain**, **ArConnect is now Wander**, **Othent is being wound down by end of 2025**, and **Turbo (ArDrive)** has become the dominant bundler — accepting fiat (Stripe), AR, ETH, SOL, POL, KYVE, ARIO, and USDC-on-Base, but **not ALGO**. For your stack the upshot is: keep Lighthouse as warm/perpetual storage, layer Arweave underneath as cryptographic-permanence cold archive, anchor everything by `bytes32` Arweave txid in an Ethereum mainnet registry, and bridge ALGO→USDC-on-Base to fund Turbo when x402 payments come in over Algorand.

---

1. What Arweave actually is — from first principles

Arweave is a **blockweave**, not a blockchain. Each new block references the immediately previous block *and* a deterministically pseudo-random **recall block** drawn from anywhere in history. To produce a valid block, a miner must furnish a 256 KiB chunk from inside that recall block plus its Merkle inclusion proof; since the recall target is unpredictable, miners are economically pressed to keep as much of the weave on disk as possible. This is the consensus shape called **SPoA** (Succinct Proof of Access) in its earliest form, **SPoRA** (Succinct Proofs of Random Access) since 2021, and as of 2025 **`replica.2.9`-packing SPoRA**, where each replica is keyed by `SHA256(chunk_offset || tx_root || miner_address)` so every miner stores a *unique* representation of the same logical bytes. A VDF runs at one step per second network-wide to bound difficulty manipulation. Mining has shifted from CPU/GPU bandwidth contests to **packing-bound storage**: optimal mining read-rate is now ~**5 MiB/s per partition** under `replica.2.9` versus ~200 MiB/s under the older `spora_2_6`, so a single 16 TB HDD can host roughly forty 4 TB partitions while contributing the same effective hashrate. Partition size is **3.6 TB**; the weave was ~**373 TB / 103 partitions** in late 2025 and continues to grow. Oreate AI + 4

The **economic model** is the part that earns the slogan *"pay once, store forever."* Every upload pays a one-time AR fee that is split between immediate miner reward and the **storage endowment**, a protocol-managed pool that pays miners out gradually for replicating that data over the next ~200 years. The endowment math (Martin Kleppmann's analysis embedded in the Yellow Paper) prices the upfront fee against an assumed **30.5%/year storage cost decline** (Kryder's law); under conservative cost-decline scenarios the endowment is sized to outlast two centuries. The native token is hard-capped at **66,000,000 AR**; circulating supply in May 2026 is ~**65,652,466 AR (~99.5% diluted)**, so block-reward emissions are now numerically minor — miner income is

increasingly *endowment*-funded rather than emission-funded, which is exactly what the model predicted. CoinMarketCap + 3

Is Arweave an EVM? Definitively, **no**. There is no Ethereum Virtual Machine, no `eth_call`, no Solidity, no global gas market. The native contract story has two generations. **SmartWeave** (2020–2024) — JavaScript contracts using **lazy evaluation**: state is computed *client-side* by replaying a contract's tagged interaction-history through a deterministic `handle(state, action)` function. The chain stores ordered inputs, not state; readers converge by re-running the same code. **AO** (2024–present) is the current, idiomatic native compute layer: each "smart contract" is an **autonomous process** with its own WASM linear memory, message inbox, and Lua VM, and the network is built from three economically-separate roles — **Messenger Units (MU)**, **Scheduler Units (SU)**, **Compute Units (CU)** — that route, order, and execute messages. AO uses Arweave as its data layer: every message and assignment is permanently archived as ANS-104 data items. GitHub + 6

The user-facing layer is the **permaweb** — the human-readable web served from Arweave through HTTP gateways. The default public gateway is `arweave.net`; **AR.IO** is the decentralized gateway network (mainnet launched February 2025) where operators stake the **ARIO** token (rebranded from "IO" pre-mainnet, fixed supply **1,000,000,000**, ~595–600M circulating in May 2026) to run incentivized gateways that index transactions, cache data, and resolve **ArNS** names like `bankon.arweave.net`. Coinbase + 2

2. Everything that changed since 2022

AO is the marquee development. Public testnet shipped February 2024; **mainnet went live February 8, 2025**. The token economics are unusually clean: **21,000,000 max supply**, Bitcoin-style halving cadence (roughly 1.425% of remaining supply minted monthly, mints emitted every 5 minutes), **100% fair launch**, no VC, no premine, no team allocation. From Feb 27, 2024 forward 100% of mints went pro-rata to AR holders; from mainnet onward the split is **33.3% to AR holders, 66.6% to bridged-asset depositors** (initially stETH and DAI, expanding to additional PoS assets). The SDK is `@permaweb/aoconnect` (`spawn`, `message`, `result`, `results`, `dryrun`, `monitor`, `createSigner`); **AO trades around $2.66–$3.26** with circulating supply ~6.1M of 21M cap. PANews + 9

HyperBEAM is the Erlang/OTP production implementation of the **AO-Core** (Converge) protocol. Where the original 2024 reference architecture ran MU/SU/CU as three Node.js services, HyperBEAM collapses all of them into a single node with **pluggable Devices**: `~process@1.0` runs an AO process, `~wasm64@1.0` executes WASM, `~scheduler@1.0` is the SU function, `~snp@1.0` provides AMD SEV-SNP TEE-attested compute, and crucially **`~patch@1.0`** lets a process publish state slices that any client can read with a plain `GET /<process>/now/cache/<key>` — the new replacement for `dryrun` round-trips. As of May 2026 HyperBEAM is in **active live-migration alongside legacy AO-net**, with payment-relay devices

bridging high-value traffic; treat any HyperBEAM-only path as fluid and verify against your target gateway before production cutover.

Turbo / Turbo Credits (ArDrive) became the dominant Arweave bundler after Bundlr/Irys pivoted away. Pricing in `@ardrive/turbo-sdk` v1.40+ is **dynamic and pegged to live AR network price** with a Turbo service margin (commonly cited around 23.4% on Stripe fiat purchases). Supported funding: **AR, ARIO, base-ARIO, ETH, SOL, POL/MATIC, KYVE, base-ETH, ETH-mainnet USDC, base-USDC, plus fiat via Stripe** (USD/EUR/GBP/CAD/AUD/JPY/INR/SGD/HKD/BRL). **Algorand is not in the list.** Uploads under 100 KiB are free. Effective USD/GiB sits in a wide band — triangulating CMC AI ($0.48/GB at AR ~$5.63), the live calculator at `ar-fees.arweave.net`, ArDrive's own widget, and Akord's published $9–$12/GB — somewhere around **$2–$9/GiB** is realistic in May 2026 depending on AR spot. Use `turbo.getFiatEstimateForBytes(bytes,"usd")` at upload time; never hard-code. FitGap + 6

ANS-104 bundling is the dominant write path: many signed DataItems wrap into a single base-layer transaction, allowing third-party payment delegation (signer ≠ payer) and high throughput. Effectively all user-facing Arweave traffic in 2026 flows through ANS-104 bundlers, and Turbo handles the lion's share post-Irys-pivot. GitHub

Irys (formerly Bundlr Network) rebranded in October 2023 and **pivoted off Arweave in 2025**: testnet January 2025, mainnet + IRYS token TGE November 25, 2025 (Binance Alpha, Coinbase, Bithumb listings). Irys is now its own EVM-compatible L1 datachain (**IrysVM**) with its own validators, a multi-ledger architecture (Submit Ledger → Publish Ledger), a hybrid useful-PoW/Stake consensus called Matrix Packaging, and cross-chain payments accepted in ETH, SOL, AVAX, APT, **ALGO**, and Berachain. Their marketing claims **~$0.05/GiB permanent**, "16× cheaper than Arweave." The strategic implication for you: **Irys is no longer an Arweave-anchoring bundler; it's a competitor.** Legacy Bundlr-era data is still on Arweave, but new Irys data lives on Irys's own ledger. If you want Arweave permanence in 2026, use Turbo, not Irys. Medium + 3

AR.IO and ArNS went mainnet February 2025. ArNS uses dynamic pricing (`Base Registration Fee × character-length multiplier × Demand Factor`), with both lease (1–5 yr) and permabuy options, undernames priced by name length, a 20% discount for healthy gateway operators, a 2-week renewal grace period, and a Returned Name Premium that decays linearly on recently-expired names. Live calculator: `arns.app`. Ar + 3

ArDrive ships the consumer file-storage UX (web, CLI, mobile, Turbo) on top of **ArFS v0.12**, the Arweave File System standard — Drives/Folders/Files/Snapshots, each with v4 UUIDs, JSON metadata in tags, public or password-encrypted private mode. **Wander** is what ArConnect rebranded to in mid-2025 — same injected `window.arweaveWallet` API surface (the `arconnect` npm types package still ships for backward compat), with multi-wallet management, AO assets and NFTs, on-wallet automation Agents, and **Wander Connect** which absorbs the social-login

niche. **Othent**, the original Auth0/Google-KMS social-login wallet, is **end-of-life by close of 2025**; new integrations should target Wander Connect. **Beacon** is the active mobile-first wallet (iOS/iPadOS/macOS), with shared wallets and an **AO-Sync** signing protocol so private keys never leave the device. npm + 5

The other primitives worth knowing: **Permaswap** is the native AO-era DEX (formerly built on everPay, now AO-native; AR/wUSDC, wAR/AOCRED, AO/wAR pairs); **Everpay** is the cross-chain real-time settlement layer (sub-second finality on its SCP ledger, supporting ETH, AR, email-derived accounts) and the substrate behind both Permaswap and the wAR ERC-20; **AOX** is everVision's MPC-secured AO bridge for AR↔AO, ETH↔AO, BSC↔AO; **ANS-110** is the asset-discoverability metadata standard (`Title`, `Type`, `Description`, `Topic:*`) that makes Arweave content GraphQL-queryable; **AOS 2.0.9** is the current Lua shell (token blueprints, `forward.computer` Hyper-AOS integration, hyperbeam state-cache). Medium + 5

The protocol upgrade cadence: **2.6** (2023, two-recall-range packing, 3.6 TB partitions, RandomX scratchpad, https://2-6-spec.arweave.net), **2.7.0** (Oct 2023, flexible Merkle tree combinations), **2.7.1** (Dec 2023, VDF retarget oscillation fix), **2.7.2** (Mar 2024, coordinated/pool mining built in), **2.8.0** (Nov 2024, composite packing `composite.1`–`composite.32` for slower-but-larger-disk mining), **2.9** (Jan 2025, `replica.2.9` packing with split entropy/packing pipeline and RandomXSquared, drastically reducing honest-miner cost), and **2.9.4.1** / **2.9.5-alpha** (2025 bug-fixes plus the `vdf hiopt_m4` Apple-Silicon-optimized VDF). No protocol-level exploits or chain reorgs were publicly reported through May 2026; the only consensus-relevant operational hiccup was the April 2025 `replica.2.9` zero-hashrate bug at block 1,642,850, fixed in 2.9.4.1. GitHub + 4

3. Permaweb pricing — the actual economics

The numbers below assume AR ≈ **$2.00**, ARIO ≈ **$0.0028**, FIL ≈ **$1.00**. Treat any Turbo $/GiB figure as a **runtime variable**, not a constant — fetch live with `turbo.getFiatEstimateForBytes(bytes,"usd")`.

Platform	$/GiB (one-time or 5-yr)	Notes
Arweave direct (AR-paid)	**~$2.00**	One-time, permanent. Range $1.40–$2.90 with AR=$1.64–$2.40
Arweave via Turbo Credits	**~$3.00 (range $2.50–$9)**	Fiat onramp + ~23.4% service fee; ardrive.io widget implies ~$3.30/GiB at upper-AR
Irys L1 permanent (NOT Arweave)	~$0.05	Native IRYS chain, not Arweave-anchored
Filecoin SP deal (~5 yr)	~$0.25–$1.00	Term, not permanent
Lighthouse perpetual (Filecoin endowment)	~$2–$5 (legacy quote)	Endowment-economic, not protocol permanence
IPFS via Pinata (~5 yr)	~$9	Subscription pinning
0G Storage	no public $/GB	Aristotle Mainnet Sep 2025 Cryptopolitan
AWS S3 Standard (5 yr)	~$1.38	Plus retrieval / egress
AWS S3 Glacier Deep Archive (5 yr)	~$0.06	12–48 hr restore, 180-day min

For practical sizing, a **10 MB LoRA** costs roughly $0.02 direct or $0.03 Turbo (free under 100 KiB); a **1 GB dataset** is ~$2 direct, ~$3 Turbo, ~$1.38 S3 Standard / 5 yr, ~$0.06 Glacier / 5 yr; a **100 GB training corpus** is ~$200 direct / ~$300 Turbo / ~$138 S3-5yr / ~$5.94 Glacier-5yr; a **1 TB checkpoint+dataset bundle** is ~$2,050 direct / ~$3,070 Turbo / ~$1,413 S3-5yr / ~$60.83 Glacier-5yr. The break-even between Arweave Turbo and S3 Standard sits around 26 months — past that, Arweave wins forever; before it, S3 wins. **AO compute has no public $/CU** in May 2026; pricing is a peer-to-peer market between CU operators and message senders, and most public processes (aoLink, Permaswap, ArNS contracts) ride on grant-subsidized CUs. Arweave

ArNS pricing in ARIO terms (translated to USD at ARIO ≈ $0.0028): a **5-character name** runs ~$7 for a 1-year lease, ~$28 for a 5-year lease, ~$84 permanent; **10-character** is ~$1.70 / $6.70 / $21; **15+ character** is ~$0.56 / $2.24 / $7. Undernames cost ~$0.14–$1.40 each. Recommendation for `bankon_ar` and friends: **lease+renew unless brand-critical**, since 1-year leases are essentially trivial expense.

4. Native Arweave smart contracts — the full story

SmartWeave is the predecessor model and is now functionally end-of-life — the awesome-ao

registry labels it "the defunct contract standard, the predecessor to AO," and AR.IO migrated its core registry from SmartWeave→AO in mid-2024. It is still useful to understand because legacy assets execute through it. A SmartWeave contract is two transactions: a JS file exporting `handle(state, action)` and an initial-state JSON. Every "interaction" is just a tagged transaction carrying an `Input`. Readers fetch all interactions via GraphQL, replay them through `handle` in a sandboxed JS engine, and converge on state. A canonical PST: GitHub Ar

```js
// (c) 2026 BANKON — all rights reserved
// SPDX-License-Identifier: Apache-2.0
export function handle(state, action) {
  const balances = state.balances, input = action.input, caller = action.caller;
  if (input.function === 'mint') {
    if (caller !== state.owner) throw new ContractError('Only the owner can mint.');
    const qty = input.qty;
    if (!Number.isInteger(qty) || qty <= 0) throw new ContractError('qty must be a p
    balances[caller] = (balances[caller] || 0) + qty;
    return { state };
  }
  if (input.function === 'transfer') {
    const { target, qty } = input;
    if (!target || target === caller) throw new ContractError('Bad target.');
    if (!Number.isInteger(qty) || qty <= 0) throw new ContractError('Invalid qty.');
    if ((balances[caller] || 0) < qty) throw new ContractError('Insufficient balance
    balances[caller] -= qty; balances[target] = (balances[target] || 0) + qty;
    return { state };
  }
  if (input.function === 'balance') {
    const target = input.target || caller;
    return { result: { target, ticker: state.ticker, balance: balances[target] || 0
  }
  throw new ContractError(`Unknown function: ${input.function}`);
}
```

AO Processes are the modern path. Each process is a sovereign actor: ANS-104 inbox, Lua-on-WASM execution, holographic state log on Arweave, scheduled by an SU and executed by a CU. Processes communicate only via **signed messages** — no shared memory, no global VM. The `Action` tag is the conventional dispatch key; `Anchor` is a 32-byte client nonce for replay protection; **cron** is just synthetic `Action=Cron` messages injected at a chosen interval. The **CU/MU/SU** roles separate: an MU edge-routes user messages to the right SU and "cranks" outbox messages out to their target processes; the SU assigns each message a strictly-increasing per-

process slot and bundles it into an ANS-104 data item that gets uploaded to Arweave so the canonical order is permanent and publicly auditable; the CU pulls messages in slot order, runs them through the WASM module, and returns `Result = { Messages, Spawns, Output, Error }` — multiple CUs can independently re-execute and converge, with disagreement provable via signed outputs. HyperBEAM collapses this into a single Erlang/OTP node with pluggable devices but the protocol-level abstraction is unchanged. DEV Community + 13

A production-grade AO token contract (idiomatic Handlers pattern, big-int math, ao Standard Token notices):

```lua
```

```
-- (c) 2026 BANKON — all rights reserved
-- SPDX-License-Identifier: Apache-2.0
local bint = require('.bint')(256)
local json = require('json')
Name=Name or 'BANKON'; Ticker=Ticker or 'BKN'; Denomination=Denomination or 12
Balances = Balances or { [ao.id] = tostring(bint(10) ^ bint(20)) }
TotalSupply = TotalSupply or tostring(bint(10) ^ bint(20))
local U = { add=function(a,b)return tostring(bint(a)+bint(b))end,
            sub=function(a,b)return tostring(bint(a)-bint(b))end,
            gte=function(a,b)return bint(a)>=bint(b)end,
            gt0=function(a)return bint(a)>bint(0)end }
Handlers.add('Info',{Action='Info'},function(m)
  m.reply({Name=Name,Ticker=Ticker,Denomination=tostring(Denomination),TotalSupply=T
end)
Handlers.add('Balance',{Action='Balance'},function(m)
  local t=m.Tags.Recipient or m.Tags.Target or m.From
  m.reply({Action='Balance-Notice',Balance=Balances[t] or '0',Ticker=Ticker,Account=
end)
Handlers.add('Transfer',{Action='Transfer'},function(m)
  assert(type(m.Tags.Recipient)=='string','Recipient required')
  assert(type(m.Tags.Quantity)=='string' and U.gt0(m.Tags.Quantity),'Quantity requir
  local f,t,q=m.From,m.Tags.Recipient,m.Tags.Quantity
  Balances[f]=Balances[f] or '0'; Balances[t]=Balances[t] or '0'
  if not U.gte(Balances[f],q) then m.reply({Action='Transfer-Error',Error='Insuffici
  Balances[f]=U.sub(Balances[f],q); Balances[t]=U.add(Balances[t],q)
  if not m.Tags.Cast then
    local d={Target=f,Action='Debit-Notice',Recipient=t,Quantity=q,Data='Debited '..
    local c={Target=t,Action='Credit-Notice',Sender=f,Quantity=q,Data='Credited '..q
    for k,v in pairs(m.Tags) do if k:sub(1,2)=='X-' then d[k]=v; c[k]=v end end
    m.reply(d); Send(c)
  end
end)
Handlers.add('Mint',{Action='Mint'},function(m)
  assert(m.From==Owner or m.From==ao.id,'Only Owner can mint')
  assert(type(m.Tags.Quantity)=='string' and U.gt0(m.Tags.Quantity),'Quantity requir
  Balances[Owner]=U.add(Balances[Owner] or '0',m.Tags.Quantity)
  TotalSupply=U.add(TotalSupply,m.Tags.Quantity)
  m.reply({Action='Mint-Notice',Quantity=m.Tags.Quantity,TotalSupply=TotalSupply})
end)
```

A capability-indexed agent registry (the right shape for AgenticPlace agent discovery):

lua

```lua
-- (c) 2026 BANKON — all rights reserved
-- SPDX-License-Identifier: Apache-2.0
local json = require('json')
Agents = Agents or {}; CapIdx = CapIdx or {}; OwnerIdx = OwnerIdx or {}
local function setKeys(t) local o={} for k,_ in pairs(t or {}) do o[#o+1]=k end retu
local function idxAdd(idx,k,id) idx[k]=idx[k] or {}; idx[k][id]=true end
local function idxRm(idx,k,id) if idx[k] then idx[k][id]=nil if next(idx[k])==nil th
Handlers.add('Register',{Action='Register'},function(m)
  assert(type(m.Tags.Name)=='string'); assert(type(m.Tags.Capabilities)=='string')
  local ok,caps=pcall(json.decode,m.Tags.Capabilities); assert(ok and type(caps)=='t
  local id=m.From; local owner=m.Tags.Owner or m.From
  if Agents[id] then for _,c in ipairs(Agents[id].capabilities) do idxRm(CapIdx,c,id
                     idxRm(OwnerIdx,Agents[id].owner,id) end
  Agents[id]={id=id,name=m.Tags.Name,owner=owner,capabilities=caps,
              endpoint=m.Tags.Endpoint or '',registeredAt=m.Timestamp or os.time()}
  for _,c in ipairs(caps) do idxAdd(CapIdx,c,id) end; idxAdd(OwnerIdx,owner,id)
  m.reply({Action='Register-Notice',AgentId=id,Data=json.encode(Agents[id])})
end)
Handlers.add('FindByCapability',{Action='FindByCapability'},function(m)
  local ids=setKeys(CapIdx[m.Tags.Capability]); local out={}
  for _,id in ipairs(ids) do out[#out+1]=Agents[id] end
  m.reply({Action='Query-Result',Capability=m.Tags.Capability,Count=tostring(#out),D
end)
Handlers.add('List',{Action='List'},function(m)
  local cur=tonumber(m.Tags.Cursor) or 1; local lim=math.min(tonumber(m.Tags.Limit)
  local ids=setKeys(Agents); table.sort(ids); local page={}
  for i=cur,math.min(#ids,cur+lim-1) do page[#page+1]=Agents[ids[i]] end
  m.reply({Action='List-Result',Total=tostring(#ids),
           Next=cur+lim<=#ids and tostring(cur+lim) or '',Data=json.encode(page)})
end)
```

Cron-driven autonomous agents simply add an `Action=Cron` handler and tag the spawn with `Cron-Interval=1-minute`; calling `.monitor` in `aos` or `monitor()` in `aoconnect` starts the MU-side worker that actively cranks scheduled messages. npm

Deploy interactively with the **`aos`** CLI: `npm i -g https://get_ao.g8way.io`, then `aos sweeper --load token.lua --load registry.lua --cron 1-minute --tag-name App-Name --tag-value BANKON --wallet ./wallet.json`. Programmatically, **`@permaweb/aoconnect`** drives spawn/message/result/dryrun/monitor through `createSigner(jwk)`; verify the current `AOS_MODULE` and `SCHEDULER` IDs at `cookbook_ao.arweave.net` before mainnet because module IDs roll forward as new aos versions ship. In HyperBEAM-era code, expose state via

`Send({device='patch@1.0', cache={balances=Balances}})` and read it from any client with `curl https://<hb-node>/<processId>/now/cache/balances` — no signer, no dryrun, no CU round-trip.

5. Acquiring AR from decentralized sources

The Arweave on-ramp problem is real: liquidity is thin and the bridges are not commodity infrastructure. The verified contract addresses you need:

- **wAR on Ethereum:** `0x4fadc7a98f2dc96510e42dd1a74141eEae0C1543` — Etherscan-verified, deployed July 2021 by everFinance, **decimals = 12** (matches Arweave's winston-equivalent precision, *not* 18). Primary pool is WAR/WETH on Uniswap V3 at `0x3afec5673a547861877f4d722a594171595e561b` (0.3% tier) — liquidity is low four-figure USD, so split anything over ~$200 into chunks and set ⩾3% slippage. Etherscan GeckoTerminal
- **wAR on BSC:** `0x7209331e2f3B4Bb86ac1EaD771e81fFae1dDeE8D` — BscScan-verified, decimals=12, includes the canonical `event Burn(address sender, string wallet, uint256 amount)` and `burn(uint256 amount, string memory wallet)` payable function (burnCost 0.003 BNB) used by everPay's withdraw-to-Arweave path. **Avoid the imposter** at `0x00b11c87e4c4563f698c32fd28dc7176e1529b4c` (unverified, decimals=18, supply 1e15). BscScan BscScan
- **everPay ethLocker** (the Ethereum escrow): `0x38741a69785e84399fcf7c5ad61d572f7ecb1dab` (PeckShield audited). Medium

The two practical paths from ETH on mainnet are: **(A)** Uniswap V3 directly into wAR (only viable under ~$200 due to thin liquidity), then deposit wAR into everPay, then withdraw to Arweave native AR (Watchmen sign, ArLocker releases, ~30–60 min round trip). **(B)** The deeper-liquidity path: deposit ETH directly into everPay → swap ETH → wUSDC inside everPay (zero-gas, virtual-pool match) → withdraw wUSDC into AO via AOX → swap wUSDC → wAR on **Permaswap** (the deepest AO-side wAR pool) → withdraw wAR → native AR through everPay/AOX. Both Bundlr/Irys top-ups (paying in ETH/SOL/MATIC/AVAX/APT/**ALGO**/Berachain/Linea/Base) and Turbo top-ups (AR/ARIO/ETH/SOL/POL/KYVE/base-eth/base-USDC/fiat) sidestep AR acquisition entirely if your goal is uploads, not custody — you never hold AR, you just buy storage credits. CEX fallbacks (Binance, KuCoin, Gate.io, OKX) all list AR/USDT spot and let you withdraw native AR directly to a 43-character address.

Bridging Algorand to Arweave has no direct audited bridge in May 2026. AOX supports AR↔AO, ETH↔AO, BSC↔AO; Astro Quantum adds Base; Vento adds Arweave↔Ethereum — none list Algorand. The practical multi-hop is **ALGO → Ethereum or Base via Wormhole or Allbridge → wAR via everPay/Uniswap → native AR.** The single-step alternative is **Irys**, which accepts ALGO directly as a payment token for Arweave-bundled uploads — but in 2026 those

uploads land on Irys's L1, not Arweave proper, so use Irys only when its permanence model is acceptable. Arweave Hub

6. Bridging architecture

Everpay is the most mature Arweave bridge and the substrate behind the wAR ERC-20. Its layered design: source-chain locker contracts (`ethLocker` on Ethereum, BSC equivalent) escrow native and ERC/BEP-20 assets; `ArLocker` on Arweave runs a Safeheron-TEE-backed threshold signature scheme since Arweave has no on-chain multisig; an off-chain SCP smart contract run by Coordinator + Watchmen + Detectors produces the canonical ledger by reading transactions from Arweave (storage as source of truth); a ProposalHub collects ⅔ or ⅗ Watchman signatures and executes proposals. Withdrawals burn wAR (calling ERC-20 `_burn` and emitting `Burn(...)` with the target Arweave address) → Watchmen verify → ArLocker releases AR. **AOX**, built atop everPay, replaced classical multisig with MPC signing in April 2024 and is the ecosystem's primary vehicle for AR↔AO, ETH↔AO, and BSC↔AO movements (USDT bridging from BSC went live January 2025). Astro's Quantum Bridge adds Base support; Vento adds an alternative Arweave↔Ethereum route. Medium + 4

For an **x402-Algorand-pays-for-Arweave-uploads** flow, the architecture is necessarily multi-hop: parsec-wallet pays in ALGO via x402 → server's payment processor verifies the Algorand transaction → bridges ALGO to USDC on Base (Wormhole guardians ~2–4 min, Allbridge ~1 min with smaller liquidity) → tops up Turbo Credits with `topUpWithTokens({token:"base-usdc"})` → `turbo.uploadFile()` returns the Arweave txid. The bottleneck is the bridge, so the production optimization is to **pre-fund Turbo with USDC-on-Base in bulk and settle the Algorand→USDC bridge asynchronously** (batch hourly), which collapses the user-visible round trip to under 10 seconds.

7. Storing data on Arweave — full code in four languages

Python (≥3.12), using `arweave-python-client==1.0.19` (maintenance-only; last release Oct 2024 but still functional) plus `gql`, `httpx`, `cryptography`, with the JWK generated from RSA-4096 (`e=65537` is mandatory for valid Arweave signatures): Snyk Arweave

```python
```

```
# (c) 2026 BANKON — all rights reserved
# SPDX-License-Identifier: Apache-2.0
from __future__ import annotations
import base64, json, logging, os, sys
from pathlib import Path
import arweave
from arweave.arweave_lib import Transaction, Wallet
from arweave.transaction_uploader import get_uploader
from cryptography.hazmat.primitives.asymmetric import rsa
from gql import Client, gql
from gql.transport.requests import RequestsHTTPTransport

GATEWAY, GRAPHQL = "https://arweave.net", "https://arweave.net/graphql"
log = logging.getLogger("ar"); logging.basicConfig(level=logging.INFO)

def _b64url(v: int, n: int | None = None) -> str:
    raw = v.to_bytes(n if n else (v.bit_length() + 7) // 8, "big")
    return base64.urlsafe_b64encode(raw).rstrip(b"=").decode()

def generate_jwk(out: Path) -> Path:
    k = rsa.generate_private_key(public_exponent=65537, key_size=4096)
    pn = k.private_numbers(); p = pn.public_numbers
    jwk = {"kty":"RSA","e":_b64url(p.e),"n":_b64url(p.n,512),"d":_b64url(pn.d,512),
           "p":_b64url(pn.p,256),"q":_b64url(pn.q,256),"dp":_b64url(pn.dmp1,256),
           "dq":_b64url(pn.dmq1,256),"qi":_b64url(pn.iqmp,256)}
    out.write_text(json.dumps(jwk)); out.chmod(0o600); return out

def upload_text(wallet: Path, payload: str, content_type="text/plain; charset=utf-8"
                extra_tags: dict | None = None) -> str:
    w = Wallet(str(wallet))
    tx = Transaction(w, data=payload.encode())
    tx.add_tag("Content-Type", content_type)
    tx.add_tag("App-Name", "BANKON-ArweaveRef"); tx.add_tag("App-Version","1.0.0")
    for k, v in (extra_tags or {}).items(): tx.add_tag(k, v)
    tx.sign(); tx.send(); return tx.id

def upload_chunked(wallet: Path, file_path: Path, content_type="application/octet-st
    w = Wallet(str(wallet))
    with file_path.open("rb", buffering=0) as fh:
        tx = Transaction(w, file_handler=fh, file_path=str(file_path))
        tx.add_tag("Content-Type", content_type); tx.add_tag("File-Name", file_path.
        tx.add_tag("App-Name","BANKON-ArweaveRef"); tx.sign()
        u = get_uploader(tx, fh)
```

```
        while not u.is_complete: u.upload_chunk()
    return tx.id

_QUERY = gql("""query($n:String!,$v:[String!]!,$first:Int!,$after:String){
  transactions(tags:[{name:$n,values:$v}],first:$first,after:$after,sort:HEIGHT_DESC
    pageInfo{hasNextPage} edges{cursor node{id owner{address} data{size type}
      block{height timestamp} tags{name value}}}}}""")

def find_by_tag(name: str, values: list[str], page: int = 50):
    c = Client(transport=RequestsHTTPTransport(url=GRAPHQL, retries=3, timeout=30))
    after = None
    while True:
        r = c.execute(_QUERY, variable_values={"name":name,"values":values,"first":p
        for e in r["transactions"]["edges"]: yield e["node"]
        if not r["transactions"]["pageInfo"]["hasNextPage"]: return
        after = r["transactions"]["edges"][-1]["cursor"]
```

Direct Turbo HTTP from Python (no official PyPI Turbo SDK exists in May 2026 — sign an ANS-104 DataItem with `arbundles-py` and POST to `https://upload.ardrive.io/tx`):

python

```
# (c) 2026 BANKON — all rights reserved
# SPDX-License-Identifier: Apache-2.0
import httpx
from pathlib import Path
from arbundles import ArweaveSigner, DataItem  # pip install arbundles-py
from arweave.arweave_lib import Wallet
TURBO_TX = "https://upload.ardrive.io/tx"
TURBO_PRICE = "https://upload.ardrive.io/price/bytes"

def turbo_upload(wallet_path: Path, payload: bytes, tags: list[tuple[str,str]]) -> d
    w = Wallet(str(wallet_path)); s = ArweaveSigner(w.jwk_data)
    di = DataItem(data=payload, tags=[{"name":n,"value":v} for n,v in tags])
    di.sign(s); raw = di.get_raw()
    with httpx.Client(timeout=60) as cx:
        cx.get(f"{TURBO_PRICE}/{len(raw)}").raise_for_status()
        r = cx.post(TURBO_TX, content=raw,
                    headers={"Content-Type":"application/octet-stream",
                             "Content-Length":str(len(raw))})
        r.raise_for_status(); return r.json()
```

TypeScript (Node ≥20) — `arweave-js` for chunked uploads, `@ardrive/turbo-sdk` v1.40+ for the bundled path with cost preview, `@permaweb/aoconnect` for AO:

typescript

```
// (c) 2026 BANKON — all rights reserved
// SPDX-License-Identifier: Apache-2.0
import Arweave from "arweave";
import { ArweaveSigner, TurboFactory } from "@ardrive/turbo-sdk";
import { spawn, message, result, dryrun, createSigner } from "@permaweb/aoconnect/no
import { promises as fs } from "node:fs";
import { createReadStream, statSync } from "node:fs";
import type { JWKInterface } from "arweave/node/lib/wallet.js";

const arweave = Arweave.init({ host:"arweave.net", port:443, protocol:"https" });

export async function loadOrCreate(p: string): Promise<JWKInterface> {
  try { return JSON.parse(await fs.readFile(p,"utf8")); }
  catch { const j = await arweave.wallets.generate();
          await fs.writeFile(p, JSON.stringify(j), { mode:0o600 }); return j; }
}
export async function postChunked(jwk: JWKInterface, data: Uint8Array | string,
                                  tags: Record<string,string>) {
  const tx = await arweave.createTransaction({ data }, jwk);
  for (const [k,v] of Object.entries(tags)) tx.addTag(k, v);
  await arweave.transactions.sign(tx, jwk);
  const u = await arweave.transactions.getUploader(tx);
  while (!u.isComplete) await u.uploadChunk();
  return { id: tx.id, url: `https://arweave.net/${tx.id}` };
}
export async function turboUpload(jwk: JWKInterface, filePath: string,
                                  tags: {name:string;value:string}[]) {
  const turbo = TurboFactory.authenticated({ signer: new ArweaveSigner(jwk) });
  const size = statSync(filePath).size;
  const [{ winc: cost }, { winc: bal }] = await Promise.all([
    turbo.getUploadCosts({ bytes:[size] }).then(r => r[0]),
    turbo.getBalance() ]);
  if (BigInt(bal) < BigInt(cost)) throw new Error(`insufficient credits: ${bal} < ${
  return turbo.uploadFile({
    fileStreamFactory: () => createReadStream(filePath),
    fileSizeFactory:   () => size,
    dataItemOpts: { tags } });
}
export async function aoSpawnAndPing(jwk: JWKInterface) {
  const s = createSigner(jwk);
  const pid = await spawn({
    module: "ISShJH8KzrVyaHE3J4f4eHBmGZkDfgBoO5BPpHIQHWs",   // verify in cookbook_a
    scheduler: "_GQ33BkPtZrqxA84vM8Zk-N2aO0toNNu_C-l-rawrBA",
```

```
    signer: s, tags:[{name:"App-Name",value:"BANKON-ArweaveRef"}] });
  const mid = await message({ process: pid, signer: s,
    tags:[{name:"Action",value:"Eval"}],
    data: 'Handlers.add("ping","ping",function(m) Send({Target=m.From,Data="pong"})
  return { pid, evalResult: await result({ process: pid, message: mid }) };
}
```

BASH — install the toolchain once, then use canonical commands:

bash

```
#!/usr/bin/env bash
# (c) 2026 BANKON — all rights reserved
# SPDX-License-Identifier: Apache-2.0
set -euo pipefail
WALLET="${WALLET:-$HOME/.arweave/wallet.json}"; GW="https://arweave.net"
install_tooling() { npm i -g arweave-deploy@1.9.1 ardrive-cli@3.1.0 @ardrive/turbo-s
ensure_wallet()   { mkdir -p "$(dirname "$WALLET")"; [[ -f $WALLET ]] || arweave key
                    echo "Address: $(arweave key-inspect "$WALLET")"; }
deploy_file()     { arweave deploy "$1" --key-file "$WALLET" --confirm; }
ardrive_publish() { local d=$(ardrive create-drive -w "$WALLET" -n BANKON-Drive --tu
                    local f=$(echo "$d" | jq -r '.created[]|select(.type=="folder").
                    ardrive upload-file --local-path "$1" --parent-folder-id "$f" \
                            --wallet-file "$WALLET" --turbo; }
turbo_upload()    { npx -y @ardrive/turbo-sdk@1.40.2 upload-file -f "$1" \
                    --wallet-file "$WALLET" --tags Content-Type "$(file -b --mime-ty
                    App-Name BANKON-ArweaveRef; }
gql_find()        { curl -fsS "$GW/graphql" -H 'Content-Type: application/json' \
                    --data-binary @- <<EOF | jq '.data.transactions.edges[].node|{id
{"query":"query(\$n:String!,\$v:[String!]!){transactions(tags:[{name:\$n,values:\$v}
 "variables":{"n":"App-Name","v":["${1:-BANKON-ArweaveRef}"]}}
EOF
                  }
fetch_tx()        { curl -fsS -o "${1}.bin" "$GW/$1"; }
launch_aos()      { npx -y @permaweb/aos@latest "$WALLET"; }
"$@"
```

Solidity (0.8.26 + Foundry + OpenZeppelin v5.6.1) — flat snake_case layout. The contract validates 43-char base64url txids via on-chain decode, stores 32-byte raw txids with monotonic versioning, exposes pause/revoke admin paths, and emits indexed events for off-chain indexing. Project root `arweave-registry/`, `src/arweave_registry.sol`:

solidity

```solidity
// SPDX-License-Identifier: Apache-2.0
// (c) 2026 BANKON — all rights reserved
pragma solidity 0.8.26;
import {AccessControl} from "@openzeppelin/contracts/access/AccessControl.sol";
import {Pausable} from "@openzeppelin/contracts/utils/Pausable.sol";

contract ArweaveRegistry is AccessControl, Pausable {
    bytes32 public constant PUBLISHER_ROLE = keccak256("PUBLISHER_ROLE");
    bytes32 public constant PAUSER_ROLE    = keccak256("PAUSER_ROLE");
    struct Record { bytes32 txid; uint64 publishedAt; uint32 version; address publis
    mapping(bytes32 => Record) private _latest;
    mapping(bytes32 => mapping(uint32 => bytes32)) private _history;
    event Published(bytes32 indexed contentKey, bytes32 indexed txid, uint32 indexed
                    address publisher, uint64 publishedAt);
    event Revoked(bytes32 indexed contentKey, uint32 indexed version, address by);
    error InvalidTxid(); error UnknownKey(bytes32); error UnknownVersion(bytes32, ui
    constructor(address admin) {
        _grantRole(DEFAULT_ADMIN_ROLE, admin); _grantRole(PUBLISHER_ROLE, admin); _g
    }
    function publish(bytes32 contentKey, bytes32 txid) external whenNotPaused
        onlyRole(PUBLISHER_ROLE) returns (uint32 version) {
        if (txid == bytes32(0)) revert InvalidTxid();
        Record memory prev = _latest[contentKey];
        unchecked { version = prev.version + 1; }
        Record memory rec = Record(txid, uint64(block.timestamp), version, msg.sende
        _latest[contentKey] = rec; _history[contentKey][version] = txid;
        emit Published(contentKey, txid, version, msg.sender, rec.publishedAt);
    }
    function revoke(bytes32 contentKey, uint32 version) external onlyRole(DEFAULT_AD
        if (_history[contentKey][version] == bytes32(0)) revert UnknownVersion(conte
        delete _history[contentKey][version];
        if (_latest[contentKey].version == version) delete _latest[contentKey];
        emit Revoked(contentKey, version, msg.sender);
    }
    function pause() external onlyRole(PAUSER_ROLE) { _pause(); }
    function unpause() external onlyRole(PAUSER_ROLE) { _unpause(); }
    function latest(bytes32 contentKey) external view returns (Record memory r) {
        r = _latest[contentKey]; if (r.txid == bytes32(0)) revert UnknownKey(content
    }
    function txidAt(bytes32 contentKey, uint32 version) external view returns (bytes
        t = _history[contentKey][version]; if (t == bytes32(0)) revert UnknownVersio
    }
    function gatewayUrl(bytes32 txid) external pure returns (string memory) {
```

```
        return string.concat("https://arweave.net/", _toBase64Url(txid));
    }
    function parseTxid(string calldata b) external pure returns (bytes32) {
        bytes memory s = bytes(b); if (s.length != 43) revert InvalidTxid();
        return _decode43(s);
    }
    bytes internal constant _ALPHA = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrst
    function _toBase64Url(bytes32 raw) internal pure returns (string memory) {
        bytes memory o = new bytes(43); uint256 v;
        for (uint256 i; i < 30; i += 3) {
            v = (uint256(uint8(raw[i])) << 16) | (uint256(uint8(raw[i+1])) << 8) | u
            o[(i/3)*4]=_ALPHA[(v>>18)&0x3F]; o[(i/3)*4+1]=_ALPHA[(v>>12)&0x3F];
            o[(i/3)*4+2]=_ALPHA[(v>>6)&0x3F]; o[(i/3)*4+3]=_ALPHA[v&0x3F];
        }
        v = (uint256(uint8(raw[30])) << 8) | uint256(uint8(raw[31]));
        o[40]=_ALPHA[(v>>10)&0x3F]; o[41]=_ALPHA[(v>>4)&0x3F]; o[42]=_ALPHA[(v<<2)&0
        return string(o);
    }
    function _decode43(bytes memory s) internal pure returns (bytes32 out) {
        uint256 acc; uint256 bits; uint256 written; bytes32 buf;
        for (uint256 i; i < 43; ++i) {
            uint256 d = _idx(uint8(s[i])); if (d == 0xFF) revert InvalidTxid();
            acc = (acc << 6) | d; bits += 6;
            if (bits >= 8) { bits -= 8;
                buf |= bytes32(uint256((acc >> bits) & 0xFF) << (8 * (31 - written))
        }
        if (written != 32) revert InvalidTxid(); return buf;
    }
    function _idx(uint8 c) private pure returns (uint256) {
        if (c >= 0x41 && c <= 0x5A) return c - 0x41;
        if (c >= 0x61 && c <= 0x7A) return c - 0x61 + 26;
        if (c >= 0x30 && c <= 0x39) return c - 0x30 + 52;
        if (c == 0x2D) return 62; if (c == 0x5F) return 63; return 0xFF;
    }
}
```

The Foundry test suite (`test/arweave_registry.t.sol`) should cover: event emission and storage on `publish`, monotonic version increment, revert on zero txid, AccessControl revert for unauthorized callers via `IAccessControl.AccessControlUnauthorizedAccount.selector`, paused-state revert, admin revoke + revert on unknown version, base64url roundtrip via `gatewayUrl`+`parseTxid`, revert on bad length and on URL-unsafe characters like `+`, plus a fuzz test asserting any non-zero `bytes32` round-trips. Deployment script

`script/deploy_arweave_registry.s.sol` reads `REGISTRY_ADMIN` and `DEPLOYER_PK` from env and broadcasts via `forge script ... --rpc-url mainnet --broadcast --verify`. `foundry.toml` should pin `solc = "0.8.26"`, `evm_version = "cancun"`, `optimizer_runs = 200`.

The end-to-end glue: client-side, convert a 43-char base64url txid to `bytes32` via `Buffer.from(b64url, "base64url")` and call `publish(keccak256(contentKey), txid)`. Off-chain consumers index the `Published` event to learn the canonical Arweave pointer for any content key. **Do not** try to upload to Arweave from inside an EVM contract — the EVM cannot speak HTTP to a gateway, has no RSA-PSS, and has no native Arweave signer; the EVM's job is to *commit* to txids that were produced off-chain.

8. Wallet integration including parsec

Arweave wallets are **RSA-4096 JWKs**, not secp256k1 keys. The public exponent **must** be 65537 (`"e": "AQAB"`); transaction signing uses **RSA-PSS** with SHA-256 over a Merkle root of the transaction fields (`format`, `owner`, `target`, `data_root`, `data_size`, `quantity`, `reward`, `last_tx`, `tags`); encryption uses RSA-OAEP. The address is the unpadded base64url encoding of `SHA-256(n_bytes)` — exactly **43 ASCII characters**. The full JWK structure is `{kty:"RSA", n, e, d, p, q, dp, dq, qi, ext?}` where the CRT fields enable fast private-key operations. Arweave + 2

The injected wallet API (`window.arweaveWallet`) is the Wander/ArConnect interface; apps wait for the `arweaveWalletLoaded` DOM event then call `connect(permissions, appInfo, gateway)` with permission strings drawn from `ACCESS_ADDRESS`, `ACCESS_PUBLIC_KEY`, `ACCESS_ALL_ADDRESSES`, `ACCESS_ARWEAVE_CONFIG`, `SIGN_TRANSACTION`, `ENCRYPT`, `DECRYPT`, `SIGNATURE`, `DISPATCH`, `ACCESS_TOKENS`. The signing methods include `sign(tx)`, `dispatch(tx)` (which auto-bundles via Turbo for fast finality), `signMessage`, `verifyMessage`, `signature`, and the ANS-104-specific `signDataItem({data, tags?, target?, anchor?})`. Wander rebrand kept the npm types package name `arconnect` for backward compatibility. Arconnect GitHub

The crucial fact for parsec-wallet integration: Arweave does **not** have a BIP-32 standard. BIP-32 child-key derivation is defined over secp256k1 (and to a lesser extent ed25519); RSA has no analogous formula because RSA keys are not points on an elliptic curve. This means:

1. You **cannot** derive an Arweave keypair deterministically from a 12/24-word BIP-39 seed in any cross-wallet-portable way. Every Arweave wallet is a standalone JWK that must be backed up directly.
2. "Mnemonic recovery" features in Wander and other wallets implement *wallet-specific* deterministic RSA seeding — generating the RSA key from a PRNG seeded by the mnemonic — but this is a vendor convention, not a standard. A Wander mnemonic generally cannot be imported into another Arweave wallet to produce the same address.

3. Hardware wallet support is sharply limited: secp256k1-only Ledger/Trezor models cannot natively sign RSA-PSS Arweave transactions. The workarounds (e.g. Bundlr/Irys/Turbo accepting Ethereum-key-signed DataItems and producing a "normalized address" derived from `base64url(SHA-256(eth_pubkey))`) produce a *different* address from a true RSA Arweave wallet. Ar

For **parsec-wallet** specifically, the recommended architecture is:

- **Generate a separate RSA-4096 JWK** alongside the existing secp256k1/ed25519 keys. Do **not** attempt to derive it deterministically from the cypherpunk2048 master seed in a way that pretends to be HD — there is no standard for it and any deterministic-seeding scheme you choose will be parsec-proprietary.
- If the user requires deterministic recovery, document the parsec-proprietary derivation explicitly: e.g. "RSA-4096 generated by `RSA.generate(seed=HKDF(master_seed, info='parsec/arweave/v1'), e=65537)`," and version it so future parsec releases can support multiple derivation versions side by side.
- Persist the JWK file separately (recommended `~/.parsec/arweave/<address>.jwk`, mode 0600) and treat it as a first-class backup artifact.
- Sign Arweave transactions client-side using `arweave-js` (TypeScript) or the Python `cryptography` + `arweave-python-client` stack shown in §7.
- For the x402-Algorand bridge to Turbo, see §10 — parsec-wallet signs the Algorand payment with its existing ed25519 Algorand key; the Arweave RSA key is only used to sign DataItems if parsec uploads directly rather than delegating to a server-side Turbo account.

Alternative for users who refuse to manage a separate RSA keyfile: route everything through Turbo or Irys top-ups paid in ETH/SOL/USDC/ALGO. They sign DataItems with their existing chain key (EthereumSigner, SolanaSigner, AlgorandSigner via Irys), the bundler stamps and posts to Arweave, and the resulting "Arweave address" is the normalized hash of the originating chain pubkey. This sidesteps RSA entirely at the cost of producing a deterministic-but-non-RSA Arweave identity.

9. Stack integration — AgenticPlace / mindX / BANKON

The recommendation for the Lighthouse-vs-Arweave question is **augment, not replace**. Lighthouse permanence is *economic-probabilistic* (Filecoin storage-provider deals plus a yield-funded endowment that re-renews them); Arweave permanence is *protocol-level* (one-time fee buys 200-year endowment baked into miner incentives via Proof-of-Access). For high-churn working data — agent traces, intermediate KV snapshots, encrypted private datasets with token-gating — Lighthouse is strictly cheaper and operationally already in production. For artifacts that must outlive the company — final aGLM checkpoints, BANKON identity attestations, DAIO constitutional records — the stronger guarantee matters and the AO compute layer can address

Arweave-resident data natively, which Lighthouse data cannot. The resulting tier system: **Tier 0 hot** (Kuzu for graph relations, Qdrant for vector embeddings, Meilisearch for BM25, NATS JetStream for streaming/inboxes); **Tier 1 warm** (Lighthouse pinning of working memory snapshots, intermediate training artifacts, encrypted private datasets); **Tier 2 cold** (Arweave via Turbo for final checkpoints, agent manifests, attestations, DAIO archives, batched memory-log bundles). X + 3

The **mindX append-only memory log** is the natural integration point. A bundler service consumes `mindx.memory.append` from JetStream, buffers ANS-104 DataItems until either 100 MiB, 6 hours, or 10,000 items, signs them with the agent's Arweave key (or a delegated Turbo Credit Share Approval), and submits via `@ardrive/turbo-sdk`. The resulting Arweave txid is then written back into Kuzu as a `memory_log_archived` edge and into the Postgres `arweave_anchor` table for fast lookup. Tags on each DataItem: `App-Name=mindX`, `Type=memory-log`, `Topic:agent=<agent_id>`, `Topic:session=<session_id>`, `Anchor-Time=<unix>` — all GraphQL-discoverable later.

The **aGLM-BANKON checkpoint** flow pins each release tier — edge (~2 GiB), mid (~8 GiB), flagship (~70 GiB), specialist-code (~14 GiB), specialist-think (~14 GiB), totaling ~108 GiB per family release — to Arweave with ANS-110 metadata plus custom tags `Model-Family`, `Model-Version`, `SHA-256`, `Size-Bytes`, `License=Apache-2.0+BANKON-2026`, then registers the resulting txid in the **BONAFIDE** on-chain registry on Ethereum mainnet via `registerCheckpoint(family, version, arTxid, sha256, sizeBytes)`. At Turbo's price midpoint ($P ≈ $9/GiB conservative, $3/GiB at the lower end of recent quotes), one full family release lands in the **$324–$972** range; sixteen releases (one per quarter for four years) total **$5.2k–$15.5k all-in for the entire history of all five model lines, permanent**. Compare AWS S3 Standard-IA at $0.0125/GiB-month for 108 GiB across four years: $64.80, *if you keep paying* — break-even vs S3-IA is around 60 years per checkpoint, after which Arweave is free forever. Always fetch live Turbo pricing with `turbo.getFiatEstimateForBytes(bytes,"usd")` rather than hardcoding.

AgenticPlace agent manifests ride on **ANS-110**: the manifest body is a JSON document with `agentId`, `displayName`, `description`, `version`, `publishedAt`, publisher addresses (ENS, Arweave, Algorand), a `model` block (primary `aGLM-BANKON-*` family/version/`arTxid`/sha256, plus fallbacks), a `capabilities` array (`tool.x402.payment`, `tool.arweave.read`, `tool.arweave.write.turbo`, `tool.algorand.sign`, `tool.kuzu.query`, `skill.policy.review`, etc.), the system-prompt blob's own `arTxid`, a `conclaveRole` block (cabinet, role, seat, convener), and an `endpoints` block (AXL pubkey, x402 URL, AO process ID). The wrapping Arweave transaction carries ANS-110 tags: `Content-Type: application/json`, `Type: agent-manifest`, `Title:`, `Description:`, plus `Topic:agent`, `Topic:conclave`, `Topic:bankon`, `Topic:counsellor`, `Topic:daio`, `Manifest-Version: 1`, `Agent-Id: agent.bankon.eth/<name>`, `Model-Txid:`, `License: Apache-2.0+BANKON-2026`. A GraphQL query like `transactions(tags:[{name:"Type",values:["agent-manifest"]},`

`{name:"Topic:bankon",values:["bankon"]},{name:"Topic:counsellor",values:["counsellor"]}])` then returns every BANKON counsellor manifest the network has seen, ordered by block height. GitHub

BANKON identity attestations are Arweave-resident signed JSON claims (W3C Verifiable Credential compatible) with tags `Type: identity-attestation`, `Vouch-For: <subject-arweave-addr>`, `Method: BANKON-KYC | ENS | Algo-MBR`, `Topic:bankon`, `SHA-256:<claim-hash>`. The on-chain commitment lives in the ENS subname registrar under `bankon.eth`: text records `ar.attestation = <arTxid>`, `ar.attestation.sha256 = <sha256>`, `ar.attestation.method = BANKON-KYC`. Verifiers resolve the ENS subname, fetch the txid from the gateway, verify the signature, and check the SHA-256 matches. GitHub

DAIO deployment uses the three-layer separation already implicit in your stack: **Algorand** is the constitutional layer (charter hash in note field, member roster, veto multisig, ASA governance — chosen for sub-3s deterministic finality, fixed cheap fee, MBR mechanics); **Ethereum mainnet (or Base/Arbitrum L2)** holds operational state (treasury, voting, proxy governance, executor calls — chosen for liquidity and tooling); **Arweave** is the permanent archive of record. Algorand and Ethereum events flow through indexers that batch into ANS-104 bundles via Turbo; the resulting `arTxid` is then committed *back* to both source chains as an anchor (Algorand note field, Ethereum log event), giving DAIO bidirectional verifiability — any chain can prove what it accepted, and Arweave proves the complete narrative.

CONCLAVE on AO is a clean fit: the 1-Convener-7-Counsellors topology maps to **8 AO processes** with the convener as orchestrator (`Action=Convene` fans out to all seven counsellors, awaits 5/7 quorum on `Action=Opinion` replies, synthesizes, returns `Action=ConclaveDecision`) and counsellors as specialized actors (Risk, Code, Compliance, Treasury, Research, Comms, Ethics). Every inter-process message is an ANS-104 DataItem persisted to Arweave by the SU, so the entire deliberation is **deterministically replayable** — exactly what a constitutional cabinet needs. The tradeoffs versus Gensyn AXL: AXL is a P2P encrypted mesh (Yggdrasil + gvisor/tcp + `localhost:9002` HTTP API) with built-in MCP and A2A support, and a `gensyn-train` CLI that uses x402 micropayments per training round; AXL latency is sub-100ms versus AO's 10–30s per round, but AXL has no shared ledger and no audit trail. The right answer is **both**: AO for the auditable governance loop, AXL as a low-latency side-channel for intra-counsellor scratchpad work and distributed inference. AO is the source of truth; AXL is ephemeral. Gensyn npm

The **chainmap** (`allchain` v2) — since `agenticplace.pythai.net/allchain.html` could not be fetched, the proposed structure to incorporate Arweave as a non-EVM permanent-storage layer:

yaml

```yaml
# (c) 2026 BANKON — all rights reserved
# SPDX-License-Identifier: Apache-2.0
schemaVersion: 2
generated: "2026-05-08"
publisher: bankon.eth
tiers:
  evm:
    - { id: ethereum-mainnet, chainId: 1,    caip2: "eip155:1",    role: [settlement
    - { id: polygon,          chainId: 137,  caip2: "eip155:137",  role: [low-fee-st
    - { id: arbitrum-one,     chainId: 42161,caip2: "eip155:42161",role: [low-fee-st
    - { id: base,             chainId: 8453, caip2: "eip155:8453", role: [primary-x4
    - { id: moonbeam,         chainId: 1284, caip2: "eip155:1284", role: [polkadot-e
    - { id: arc-testnet,      chainId: 5042002, caip2: "eip155:5042002",
        role: [stablecoin-finance, experimental], gasToken: USDC,
        status: "testnet (May 2026); mainnet chainId TBD" }
  non-evm:
    - { id: algorand-mainnet, identifier: "algorand-mainnet",
        role: [constitutional-layer, daio-charter, x402-payment-rail-custom],
        bridges: [wormhole, allbridge] }
    - { id: arweave-mainnet,  identifier: "arweave-mainnet",
        role: [permanent-storage, ans110-discovery, ao-host],
        gateway: "https://arweave.net", indexer: "https://arweave.net/graphql",
        bridges: [everpay] }
    - { id: ao-mainnet,       identifier: "ao-mainnet",
        role: [compute, actor-processes],
        addresses: "ao-process-ids (43-char base64url)", hosts-on: arweave-mainnet }
bridges:
  everpay:   { domain: "everpay.io", chains: [arweave-mainnet, ethereum-mainnet] }
  wormhole:  { chains: [ethereum-mainnet, polygon, arbitrum-one, base, moonbeam,
                        algorand-mainnet, arc-testnet] }
  allbridge: { chains: [ethereum-mainnet, base, polygon, algorand-mainnet] }
  openBDK:   { chains: [ethereum-mainnet, base, algorand-mainnet, arweave-mainnet] }
capabilities:
  store-permanently:  { tier: arweave-mainnet,  returns: arTxid }
  store-perpetually:  { tier: lighthouse,       returns: cid }
  pay-micropayment:   { tier: base,             via: x402-cdp, asset: USDC }
  pay-constitutional: { tier: algorand-mainnet, via: x402-custom, asset: ALGO|USDCa
  govern-charter:     { tier: algorand-mainnet, returns: txid }
  govern-state:       { tier: ethereum-mainnet, returns: txid }
  compute-agent:      { tier: ao-mainnet,       returns: processId }
  identity-anchor:    { tier: ethereum-mainnet, via: ens, subname-of: bankon.eth }
```

A capability resolver — the application calls `chainmap.resolve("store-permanently")` and gets routing back, never naming Arweave directly:

typescript

```typescript
// (c) 2026 BANKON — all rights reserved
// SPDX-License-Identifier: Apache-2.0
import yaml from "js-yaml";
import { ArweaveSigner, TurboFactory } from "@ardrive/turbo-sdk";
type Capability = "store-permanently" | "store-perpetually" | "pay-micropayment"
  | "pay-constitutional" | "govern-charter" | "govern-state" | "compute-agent" | "id
export class Chainmap {
  constructor(private map: any) {}
  static fromYaml(t: string) { return new Chainmap(yaml.load(t)); }
  resolve(c: Capability) { const r = this.map.capabilities[c]; if (!r) throw new Err
  async storePermanently(bytes: Uint8Array, tags: {name:string;value:string}[], sign
    const r = this.resolve("store-permanently");
    if (r.tier !== "arweave-mainnet") throw new Error("permanence resolver misconfig
    const turbo = TurboFactory.authenticated({ signer });
    const out = await turbo.uploadFile({
      fileStreamFactory: () => bytes, fileSizeFactory: () => bytes.length,
      dataItemOpts: { tags: [...tags, { name: "License", value: "Apache-2.0+BANKON-2
    return out.id;
  }
}
```

Important verification flag: **Arc chainId 5042002 is testnet, not mainnet** (verified against chainlist.org/chain/5042002, alchemy.com/rpc/arc-testnet, drpc.org). Circle's mainnet announcement (Apr 6, 2026) confirmed mainnet will ship in 2026 with post-quantum signature support but did not publish a mainnet chainId. Tag Arc explicitly as `arc-testnet` in the chainmap until Circle publishes the mainnet identifier; do not let production traffic assume `5042002` will become mainnet.

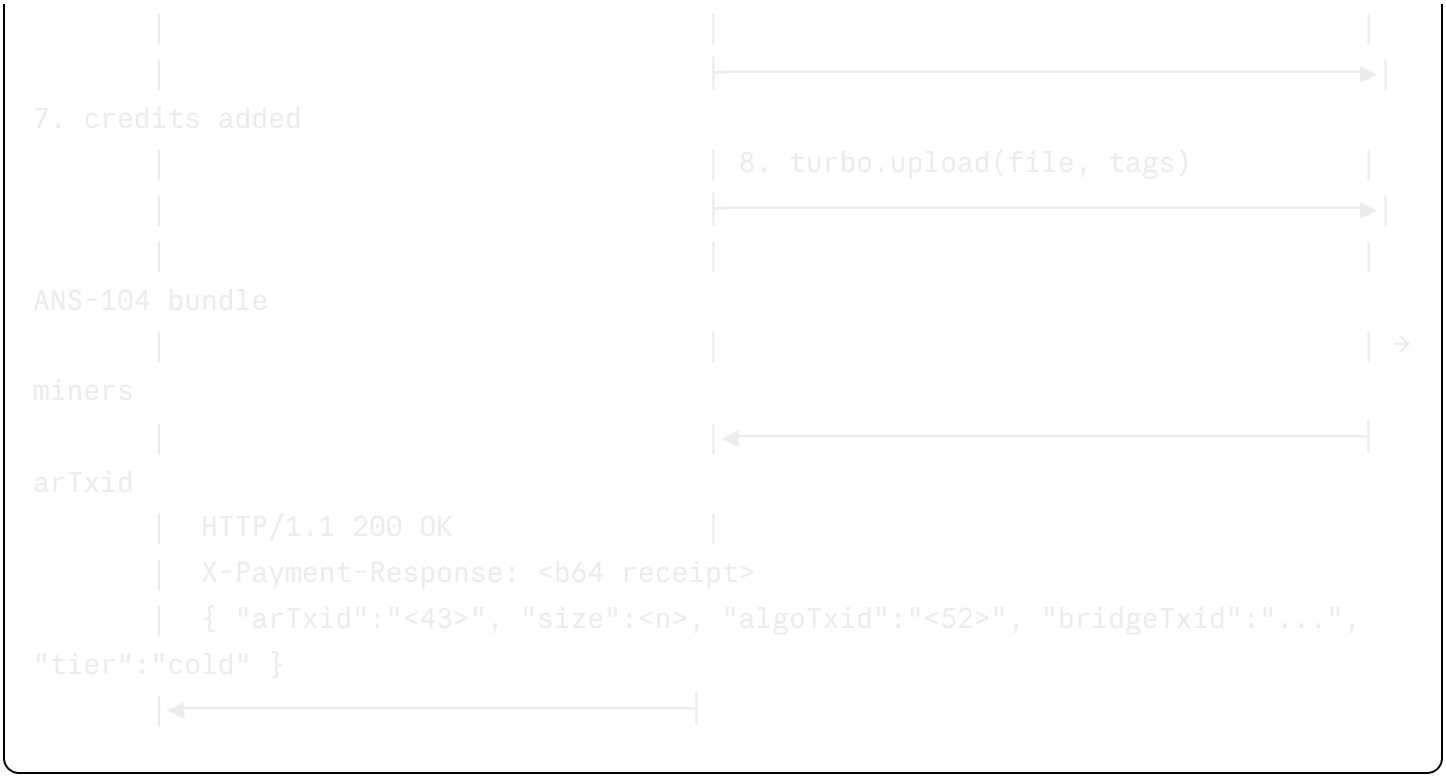
10. The x402 + Algorand + Arweave payment flow

x402 was open-sourced by Coinbase in May 2025, governance was donated to the **x402 Foundation** under the Linux Foundation on April 2, 2026, and **V2 launched December 2025/January 2026.** The HTTP flow is: client `GET /resource` → server returns **HTTP 402** with a base64-encoded `PAYMENT-REQUIRED` envelope containing `PaymentRequirements` objects (scheme, network as a CAIP-2 identifier like `eip155:8453`, payTo, asset, amount, validBefore); client picks a requirement, builds a `PaymentPayload`, signs it (EIP-3009 `transferWithAuthorization` for USDC/EURC, Permit2 for any ERC-20, SVM signing for Solana, Stellar facilitator since March 2026), retries with `PAYMENT-SIGNATURE` header; server

verifies and settles, returns `200 OK` with `X-Payment-Response`. The CDP-hosted facilitator natively settles ERC-20 on Base, Polygon, Arbitrum, World Chain, and Solana, with a Stellar facilitator since March 2026. **Algorand is not in the CDP native list**, but the x402 V2 spec is explicitly modular and supports custom facilitator plug-ins. **Turbo accepts x402 payments natively in USDC on Base** (per ar.io docs), which is the lever you should pull — instead of building an Algorand-native x402 facilitator, **bridge ALGO to USDC-on-Base and pay Turbo's native x402 endpoint**.

The end-to-end sequence:


parsec-wallet (Client)
Server / payment-processor
Algorand
Turbo / AR
POST /upload (no payment)
Content-Length: <bytes>
1. price = turbo.fiat(bytes)
2. build PaymentRequirements:
[{ scheme:"exact", network:"algorand-mainnet",
asset:"ALGO" or "USDCa",
payTo:"<ALGO addr>", amount:"<microALGO>",
validBefore: now+300,
extra:{requestHash: sha256(canon-req)} }]
HTTP/1.1 402 Payment Required
PAYMENT-REQUIRED: <b64 envelope>
3. parsec signs Algorand Pay/AssetTransfer
note = sha256(canonical request) <-- binds payment to request
POST /upload again
PAYMENT-SIGNATURE: <b64 PaymentPayload incl. signed Algo txn>
4. submit to algod
finality (<3s)
txid + confirmed-round
5. verify amount, asset, payTo,
note==sha256(request), validBefore
6. ROUTE
Turbo native list excludes ALGO →
bridge ALGO/USDCa → USDC-on-Base
via Wormhole (~2–4 min) or Allbridge (~1 min)
then
turbo.topUpWithTokens({token:"base-usdc"})



The key design move is the `note = sha256(canonical_request)` **binding** — the Li et al. preprint flags atomicity gaps in x402 V2 between off-chain verification and on-chain settlement, and embedding the request hash in the payment's note field defeats replay and front-running because any reuse of the payment receipt against a different request will fail the equality check. A FastAPI processor stub:

python

```python
# (c) 2026 BANKON — all rights reserved
# SPDX-License-Identifier: Apache-2.0
import base64, hashlib, json, os, time
from fastapi import FastAPI, Request, Response, HTTPException
from algosdk.v2client import algod
from ardrive_turbo import TurboFactory, ArweaveSigner

app = FastAPI()
algod_client = algod.AlgodClient(os.environ["ALGOD_TOKEN"], os.environ["ALGOD_URL"])
turbo = TurboFactory.authenticated(signer=ArweaveSigner(open(os.environ["AR_JWK"]).r
PAY_TO = os.environ["BANKON_ALGO_ADDR"]
PRICE_PER_GIB = float(os.environ.get("PRICE_USD_PER_GIB", "9.00"))

def request_hash(method, path, body):
    h = hashlib.sha256(); h.update(method.encode() + b"\n" + path.encode() + b"\n" +

@app.post("/upload")
async def upload(req: Request):
    body = await req.body(); size = len(body); rh = request_hash("POST", "/upload",
    sig = req.headers.get("PAYMENT-SIGNATURE")
    if not sig:
        usd = (size / 1024**3) * PRICE_PER_GIB
        algo_usd = float(os.environ.get("ALGO_USD", "0.20"))
        microalgo = int((usd / algo_usd) * 1_000_000 * 1.05)
        env = {"x402Version":2, "accepts":[{"scheme":"exact","network":"algorand-mai
               "asset":"ALGO","payTo":PAY_TO,"amount":str(microalgo),"decimals":6,
               "validBefore":int(time.time())+300,"extra":{"requestHash":rh}}]}
        return Response(status_code=402, content=json.dumps({"requirements":env}),
            headers={"PAYMENT-REQUIRED": base64.b64encode(json.dumps(env).encode()),
                     "Content-Type":"application/json"})
    payload = json.loads(base64.b64decode(sig))
    txid = algod_client.send_raw_transaction(payload["payload"]["signedTxn"])
    pending = algod_client.pending_transaction_info(txid)
    for _ in range(8):
        if pending.get("confirmed-round", 0) > 0: break
        time.sleep(1); pending = algod_client.pending_transaction_info(txid)
    if not pending.get("confirmed-round"): raise HTTPException(402, "not confirmed")
    txn = pending["txn"]["txn"]
    if txn["rcv"] != PAY_TO: raise HTTPException(402, "wrong payTo")
    if base64.b64decode(txn.get("note","")).decode(errors="ignore") != rh:
        raise HTTPException(402, "request-hash mismatch")
    bridge_txid = await bridge_algo_to_base_usdc(txid, payload["payload"]["amount"])
    await turbo.top_up_with_tokens(token_amount=payload["payload"]["amount"], token_
```

```
    receipt = await turbo.upload(data=body, data_item_opts={"tags":[
        {"name":"Content-Type","value":req.headers.get("Content-Type","application/o
        {"name":"App-Name","value":"AgenticPlace-Upload"},
        {"name":"Payer-Algo-Tx","value":txid},
        {"name":"Bridge-Tx","value":bridge_txid or ""},
        {"name":"License","value":"Apache-2.0+BANKON-2026"}]})
    return {"arTxid":receipt["id"],"algoTxid":txid,"bridgeTxid":bridge_txid,"size":s
```

Latency budget: Algorand finality ~3s (deterministic), Wormhole bridge 13 Eth-blocks ≈ 2–4 min (the bottleneck), Turbo top-up confirmation <30s on Base, Turbo upload + bundle finality 5–60s — total 3–6 minutes worst case. **The production optimization is to pre-fund Turbo with USDC-on-Base in bulk**, accept Algorand x402 payments off the critical path, settle bridging asynchronously hourly. This collapses end-to-end to under 10 seconds. Cost overhead per upload is roughly $0.0002 Algorand fee + $0.50–$2 Wormhole/Allbridge fixed fee + 0.05% volume + Turbo bundling fee (already in the credit-to-storage peg). When a Turbo Algorand-native plug-in eventually lands, the bridge disappears entirely.

Conclusion

The strategic shape of Arweave in 2026 is now clear and stable enough to commit to: **Arweave is the cryptographic-permanence storage substrate**, **AO is its native compute layer** with HyperBEAM rolling out as the production AO-Core implementation, **Turbo has won the bundling market** as Bundlr/Irys departed for its own L1, and **Wander has absorbed the wallet UX** as Othent winds down. For BANKON the integration is unambiguous: keep Lighthouse as warm/perpetual operational storage, layer Arweave underneath as cold/cryptographic permanence, anchor everything by `bytes32` Arweave txid in an Ethereum mainnet `ArweaveRegistry` contract using the OpenZeppelin v5.6.1 / solc 0.8.26 stack, route AgenticPlace agent manifests through ANS-110 for GraphQL discovery, implement CONCLAVE as 8 AO processes with AXL as a side-channel, and bridge ALGO→USDC-on-Base to fund Turbo when x402-Algorand payments arrive — pre-funded in bulk so the user-visible round trip stays under 10 seconds. The three things to verify at deployment time and never hardcode: **Turbo per-GiB pricing** (it's dynamic; `turbo.getFiatEstimateForBytes(bytes,"usd")`), **the AOS module + scheduler IDs** (they roll forward; check `cookbook_ao.arweave.net`), and **the Arc mainnet chainId** (5042002 is testnet only; mainnet is unannounced). The single most important architectural insight is the one that contradicts EVM-trained intuition: **Arweave does not run your contracts; AO runs your contracts, Arweave stores them**. Treat the two as orthogonal layers and the rest of the stack composes cleanly.